```java
package DataStructures.Trie;

public class ImplementTrie {
}

```

File - /Users/epatir/Documents/CodeWorld/codinglab/src/DataStructures/Heaps/FurthestBuildingYouCanReach.java

```java
package DataStructures.Heaps;

public class FurthestBuildingYouCanReach {
}

```

File - /Users/epatir/Documents/CodeWorld/codinglab/src/DataStructures/Maths/PalindromeNumber.java

```java
package DataStructures.Maths;

public class PalindromeNumber {
}

```

```java
package DataStructures.Arrays;

/* 283. Move Zeroes - Easy
     Given an integer array nums, move all 0's to the end of it
     while maintaining the relative order of the non-zero elements.

    Note that you must do this in-place without making a copy of the array.

    Example 1:

    Input: nums = [0,1,0,3,12]
    Output: [1,3,12,0,0]

    Complexity Analysis
        Time Complexity: O(n), where n is the length of the array.
        Space Complexity: O(1), since we are modifying the array in place without using additional storage.
*/

/*
    Cozum mantigi:

    array'i sonuna kadar dolas ve 0 olmayanlari ilk indexten baslayarak setle.
    bittikten sonra kalan indexleri 0 yap.

*/

public class MoveZeroes {
    public static void moveZeroes(int[] nums) {
        int pos = 0;

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != 0) {
                nums[pos] = nums[i];
                pos++;
            }
        }

        while (pos < nums.length) {
```

```
39                 nums[pos] = 0;
40                 pos++;
41             }
42         }
43 }
44
```

```java
package DataStructures.Arrays;

import java.util.Arrays;

public class MajorityElement {
    /* 169. Majority Element -Easy

        Given an array nums of size n, return the majority element.
        The majority element is the element that appears more than ⌊n / 2⌋ times.
        You may assume that the majority element always exists in the array.

        Example 1:
            Input: nums = [3,2,3]
            Output: 3

        Example 2:
            Input: nums = [2,2,1,1,1,2,2]
            Output: 2
    */

    /* Cozum: Boyer-Moore Voting Algorithm

        Bir mevcut elemani, bir de adeti tutan iki degisken tutariz.
        Ayni eleman geldikce artar, farkli eleman geldikce azalir.
        Butun array'i gezeriz, en sonunda candidate en fazla gecen eleman olur.
        Onu da doneriz.

        Complexity Analysis
            Time Complexity:  O(n) since it passes through the array once.
            Space Complexity: O(1) since only a few additional variables are used.

        Ya da array'i sort eder. n/2 inci elemani doneriz. Cunki majority element n/2'den daha fazla geciyor.
        Her kosulda orta eleman majority element olur.

        Complexity Analysis
```

```java
            Time Complexity: O(n log n) due to sorting.
            Space Complexity: O(1) when using in-place sorting (ignoring input space).
     */

    public static int majorityElement(int[] nums) {
        int candidate = nums[0];
        int count = 0;

        for (int num : nums) {
            if (count == 0) {
                candidate = num;
            }
            count += (num == candidate) ? 1 : -1;
        }
        return candidate;
    }

    public static int majorityElement_2(int[] nums) {
        Arrays.sort(nums);
        return nums[nums.length / 2];
    }
}

```

File - /Users/epatir/Documents/CodeWorld/codinglab/src/DataStructures/Arrays/BestTimeToBuyAndSellStock.java

```java
package DataStructures.Arrays;

public class BestTimeToBuyAndSellStock {
    public static int maxProfit(int[] prices) {
        return 0;
    }
}

```

```java
package DataStructures.Arrays;

public class RemoveDuplicatesFromSortedArray {
    /* 26. Remove Duplicates from Sorted Array -Easy

        Given an integer array nums sorted in non-decreasing order,
        remove the duplicates in-place such that each unique element appears only once.
        The relative order of the elements should be kept the same.
        Consider the number of unique elements in nums to be k.
        After removing duplicates, return the number of unique elements k.

        The first k elements of nums should contain the unique numbers in sorted order.
        The remaining elements beyond index k - 1 can be ignored.

        Example 2:

            Input: nums = [0,0,1,1,1,2,2,3,3,4]
            Output: 5, nums = [0,1,2,3,4,_,_,_,_,_]
            Explanation: Your function should return k = 5,
            with the first five elements of nums being 0, 1, 2, 3, and 4 respectively.
            It does not matter what you leave beyond the returned k (hence they are underscores).

        Complexity Analysis
            Time Complexity: O(n), where n is the number of elements in the array.
                                We traverse the array with a single pass using the two pointers.
            Space Complexity: O(1), as we are using extra space only for the pointers and directly modifying the input array.
     */

    /*
        Cozum mantigi:

```

```
32          Two pointer teknigini kullanabiliriz.Update
   edecegin indexi bir pointer olarak tut.
33          Array'i sonuna kadar dolas, writePos index'indeki
   degiskeni kiyasla.
34          eger farkli bir element yakalarsan bir index ilerle
    ve setle.
35      */
36
37      public static int removeDuplicates(int[] nums) {
38          int writePos = 0; // first pointer
39
40          for (int i = 1; i < nums.length; i++) {
41              if (nums[i] != nums[writePos]) { // compare
42                  writePos++;
43                  nums[writePos] = nums[i];
44              }
45          }
46          return writePos + 1;
47      }
48  }
49
```

```java
package DataStructures.Matrix;

public class ValidSudoku {
}

```

File - /Users/epatir/Documents/CodeWorld/codinglab/src/DataStructures/Queues/NumberOfRecentCalls.java

```java
package DataStructures.Queues;

public class NumberOfRecentCalls {
}

```

```java
package DataStructures.Stacks;

public class ValidParentheses {
}

```

```java
package DataStructures.Strings;

/*  392. Is Subsequence - Easy
        Given two strings s and t, return true if s is a subsequence of t, or false otherwise.
        A subsequence of a string is a new string that is formed from the original string
        by deleting some (can be none) of the characters without disturbing
        the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

        Example 1:

        Input: s = "abc", t = "ahbgdc"
        Output: true
        Example 2:

        Input: s = "axc", t = "ahbgdc"
        Output: false

     Complexity Analysis
        Time Complexity: O(n),  where n is the length of the array.
        Space Complexity: O(n), due to the use of additional array.
*/

/*
      Cozum mantigi:

      Two pointer teknigini kullanabiliriz.Ikinci string'i tamamen dolasiyoruz ve s string inin
      karakterini sirayla kiyasliyoruz. Bunun icin de birer pointer kullanabiliriz.
      Varsa bir pointeri ilerletiriz. Yoksa diger pointeri ilerletiriz.
*/
public class IsSubsequence {
    public static boolean isSubsequence(String s, String t) {
        if (s.isEmpty()) return true;
        if (t.isEmpty()) return false;
```

```
34
35          var sIndex = 0;
36
37          for (char tChar : t.toCharArray()) {
38              if (tChar == s.charAt(sIndex)) {
39                  sIndex++;
40                  if (sIndex == s.length()) {
41                      return true;
42                  }
43              }
44          }
45
46          return false;
47      }
48
49      public static boolean isSubsequence_2(String s, String
    t) {
50          // two pointer technique
51          int i = 0, j = 0;
52
53          // iterate through strings
54          while (i < s.length() && j < t.length()) {
55              if (s.charAt(i) == t.charAt(j)) {
56                  i++;
57              }
58              j++;
59          }
60
61          return i == s.length();
62      }
63 }
64
```

```java
package DataStructures.Strings;

public class ValidPalindrome {
    /*
        125. Valid Palindrome - Easy

        A phrase is a palindrome if, after converting all uppercase letters into lowercase letters
        and removing all non-alphanumeric characters, it reads the same forward and backward.
        Alphanumeric characters include letters and numbers.
        Given a string s, return true if it is a palindrome, or false otherwise.

        Example 1:
            Input: s = "A man, a plan, a canal: Panama"
            Output: true
            Explanation: "amanaplanacanalpanama" is a palindrome.
     */

    /*
        Complexity Analysis
            Time Complexity: O(n), where n is the length of the input string.
                                    We traverse each character twice in the worst case
                                    (once for cleaning and once for palindrome checking).
            Space Complexity: O(n) for the cleaned buffer.
    */

    public static boolean isPalindrome(String s) {

        StringBuilder cleaned = new StringBuilder();

        for (char c : s.toCharArray()) {
            if (Character.isLetterOrDigit(c)) {
                cleaned.append(Character.toLowerCase(c));
            }
        }

        // two pointers
```

```
37          int left = 0;
38          int right = cleaned.length() - 1;
39
40          // Step 3: Check palindrome property
41          while (left < right) {
42              if (cleaned.charAt(left) != cleaned.charAt(
    right)) {
43                  return false; // Not a palindrome if any
    mismatch occurs
44              }
45              left++;
46              right--;
47          }
48
49          return true;
50      }
51
52      // Two-pointers: O(1) space
53      public static boolean isPalindrome_2(String s) {
54          int left = 0;
55          int right = s.length() - 1;
56
57          while (left < right) {
58              while (left < right && !Character.
    isLetterOrDigit(s.charAt(left))) {
59                  left++;
60              }
61
62              while (left < right && !Character.
    isLetterOrDigit(s.charAt(right))) {
63                  right++;
64              }
65
66              if (Character.toLowerCase(s.charAt(left)) !=
    Character.toLowerCase(s.charAt(right))) {
67                  return false;
68              }
69
70              left++;
71              right--;
72          }
73          return true;
74      }
75  }
```

```java
package DataStructures.Strings;

import java.util.Arrays;
/* 14. Longest Common Prefix - Easy

    Write a function to find the longest common prefix string amongst an array of strings.
    If there is no common prefix, return an empty string "".

    Example 1:
        Input: strs = ["flower","flow","flight"]
        Output: "fl"
 */

/* COZUM :
           En pratik mantık (prefix'i kısaltarak gitmek)
        İlk kelimeyi prefix al.
        Diğer kelimeler prefix ile başlamıyorsa prefix'i 1 karakter kısalt.
        Prefix boşalırsa cevap "".

           Ya da string leri sort edip, ilk ve son
 */

public class LongestCommonPrefix {
    public static String longestCommonPrefix(String[] strs) {
        // TC : O(n logn)
        // SC : O(1)

        StringBuilder sb = new StringBuilder();

        // sort strings alphabetically
        Arrays.sort(strs);

        char[] firstStr = strs[0].toCharArray();
        char[] lastStr = strs[strs.length - 1].toCharArray();

        for (int i = 0; i < firstStr.length; i ++) {
            if (firstStr[i] != lastStr[i])
                break;
            sb.append(firstStr[i]);
```

```
40          }
41          System.out.println("Result: " + sb.toString());
42          return sb.toString();
43      }
44
45      // Horizontal Scanning
46      public static String longestCommonPrefix_2(String[]
    strs) {
47          // TC : O(n)
48          // SC : O(1)
49
50          if (strs == null || strs.length == 0) return "";
51
52          String prefix = strs[0];
53
54          for (int i = 1; i < strs.length; i++) {
55              while (strs[i].indexOf(prefix) != 0) {
56                  prefix = prefix.substring(0, prefix.length
    () - 1);
57
58                  // If prefix becomes empty, this means
    there is no common prefix
59                  if (prefix.isEmpty()) return "";
60              }
61          }
62          System.out.println("Result: " + prefix);
63
64          return prefix;
65      }
66
67      // Vertical Scanning
68      public static String longestCommonPrefix_3(String[]
    strs) {
69          // TC : O(n)
70          // SC : O(1)
71
72          if (strs == null || strs.length == 0) return "";
73
74          var first = strs[0];
75
76          for (int i = 0; i < first.length(); i++) {
77              char c = first.charAt(i);
78
79              for (int j = 1; j < strs.length; j++) {
```

```
80                  // If the character exceeds the length of
   one of the strings or the character doesn't match
81                  if (i == strs[j].length() || strs[j].
   charAt(i) != c) {
82                      return first.substring(0, i);
83                  }
84              }
85          }
86          System.out.println("Result: " + first);
87
88          return first;
89      }
90  }
91
```

File - /Users/epatir/Documents/CodeWorld/codinglab/src/DataStructures/HashTables/DesignHashMap.java

```java
package DataStructures.HashTables;

public class DesignHashMap {
}

```

File - /Users/epatir/Documents/CodeWorld/codinglab/src/DataStructures/LinkedLists/ListNode.java

```java
package DataStructures.LinkedLists;

public class ListNode {
    int val;
    ListNode next;
    ListNode(int x) {
        val = x;
        next = null;
    }
}
```

```java
package DataStructures.LinkedLists;

public class IntersectionOfTwoLinkedLists {
    /* 160. Intersection of Two Linked Lists - Easy
        Given the heads of two singly linked-lists headA and headB, return the node at which the two lists intersect.
        If the two linked lists have no intersection at all, return null.

    */
    public static ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        ListNode first = headA;
        ListNode second = headB;

        while (first != null) {
            System.out.println(first.val);
            first = first.next;
        }



        return null;
    }
}

```

File - /Users/epatir/Documents/CodeWorld/codinglab/src/DataStructures/MonotonicQueue/JumpGame6.java

```java
package DataStructures.MonotonicQueue;

public class JumpGame6 {
}

```

File - /Users/epatir/Documents/CodeWorld/codinglab/src/DataStructures/BinarySearchTrees/TrimABinarySearchTree.java

```java
package DataStructures.BinarySearchTrees;

public class TrimABinarySearchTree {
}

```

File - /Users/epatir/Documents/CodeWorld/codinglab/src/DataStructures/LinkedListInPlaceReversal/PalindromeLinkedList.java

```java
package DataStructures.LinkedListInPlaceReversal;

public class PalindromeLinkedList {
}

```